Machine Learning - Tutorial

Decision Trees, Random Forests

Teaching Assistant: Sourav Goswami

May 2, 2026

Decision Tree basics

DT from predictor space, and *vice versa*

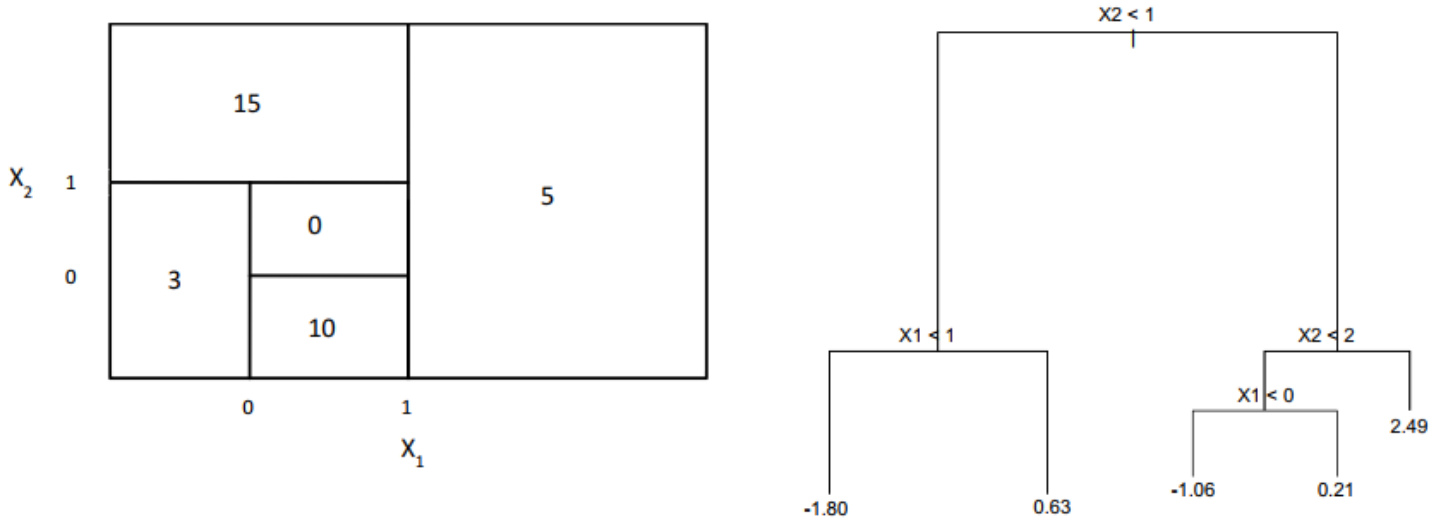


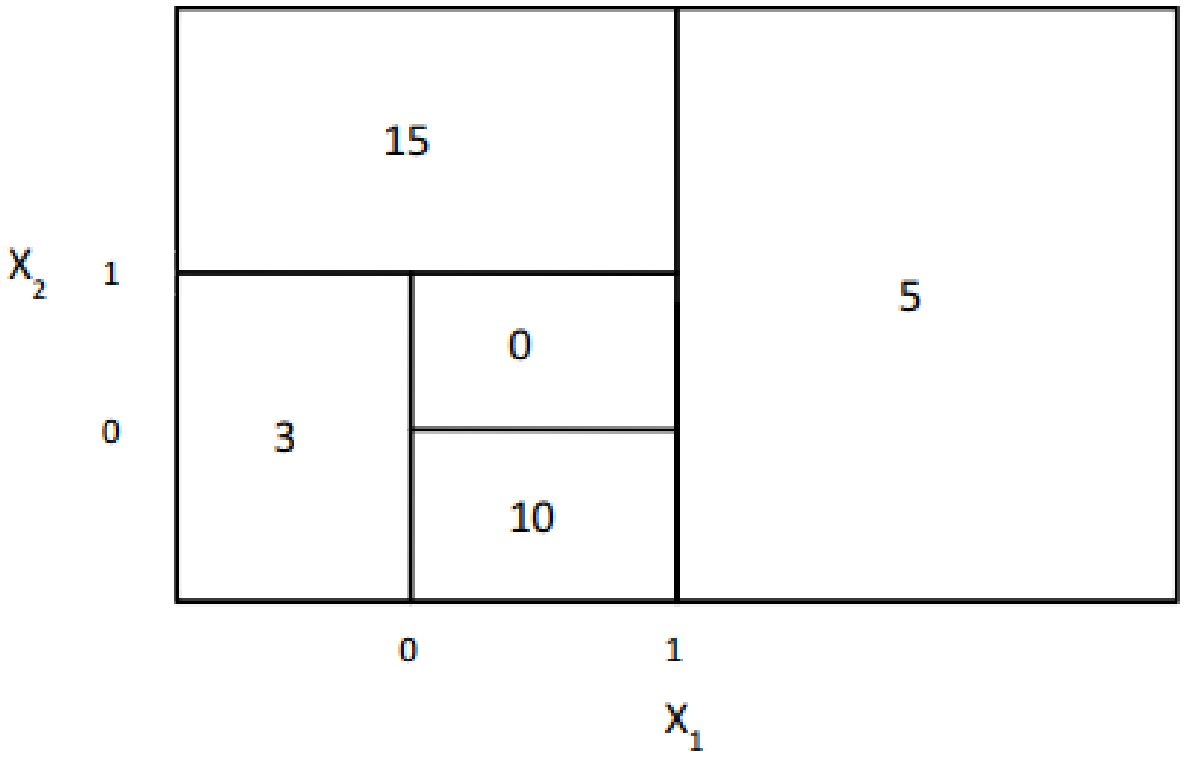
FIGURE 8.14. Left: *A partition of the predictor space corresponding to Exercise 4a.* Right: *A tree corresponding to Exercise 4b.*

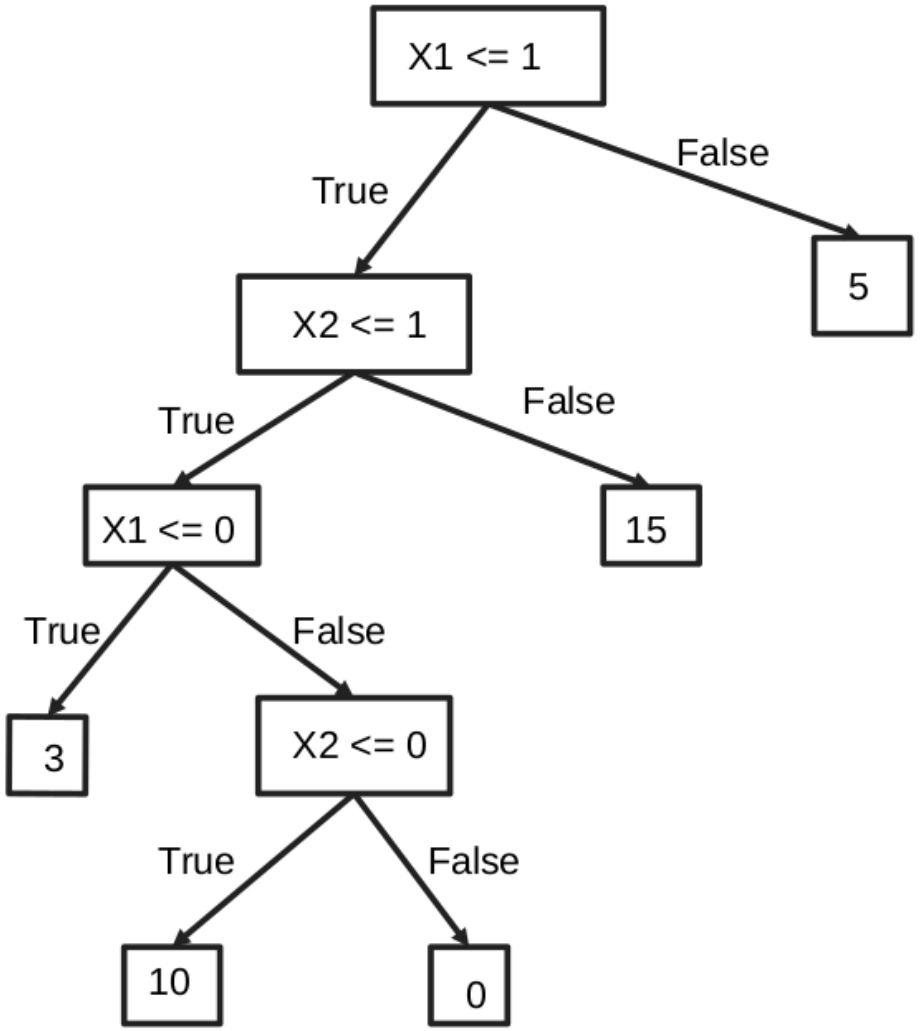
(a) Sketch the tree corresponding to the partition of the predictor space illustrated in the left-hand panel of Figure 8.14. The numbers inside the boxes indicate the mean of Y within each region.

(b) Create a diagram similar to the left-hand panel of Figure 8.14, using the tree illustrated in the right-hand panel of the same figure. You should divide up the predictor space into the correct regions, and indicate the mean for each region.

G. James et al., An Introduction to Statistical Learning, Springer Texts in Statistics, https://doi.org/10.1007/978-3-031-38747-0_9

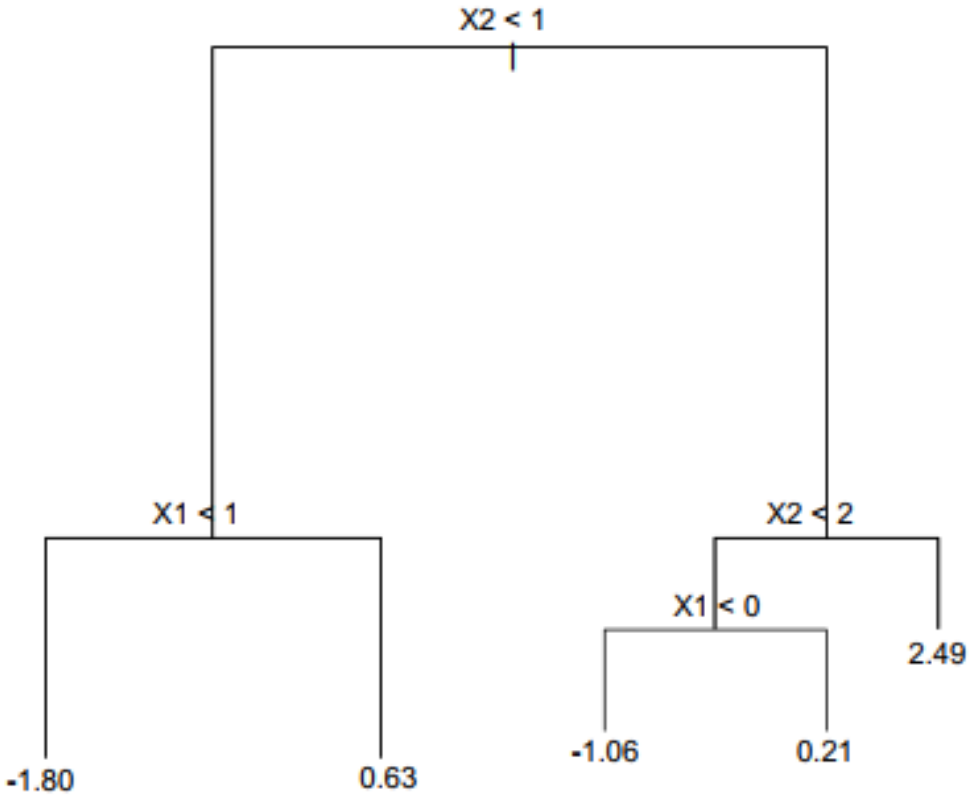
DT from predictor space, and *vice versa*

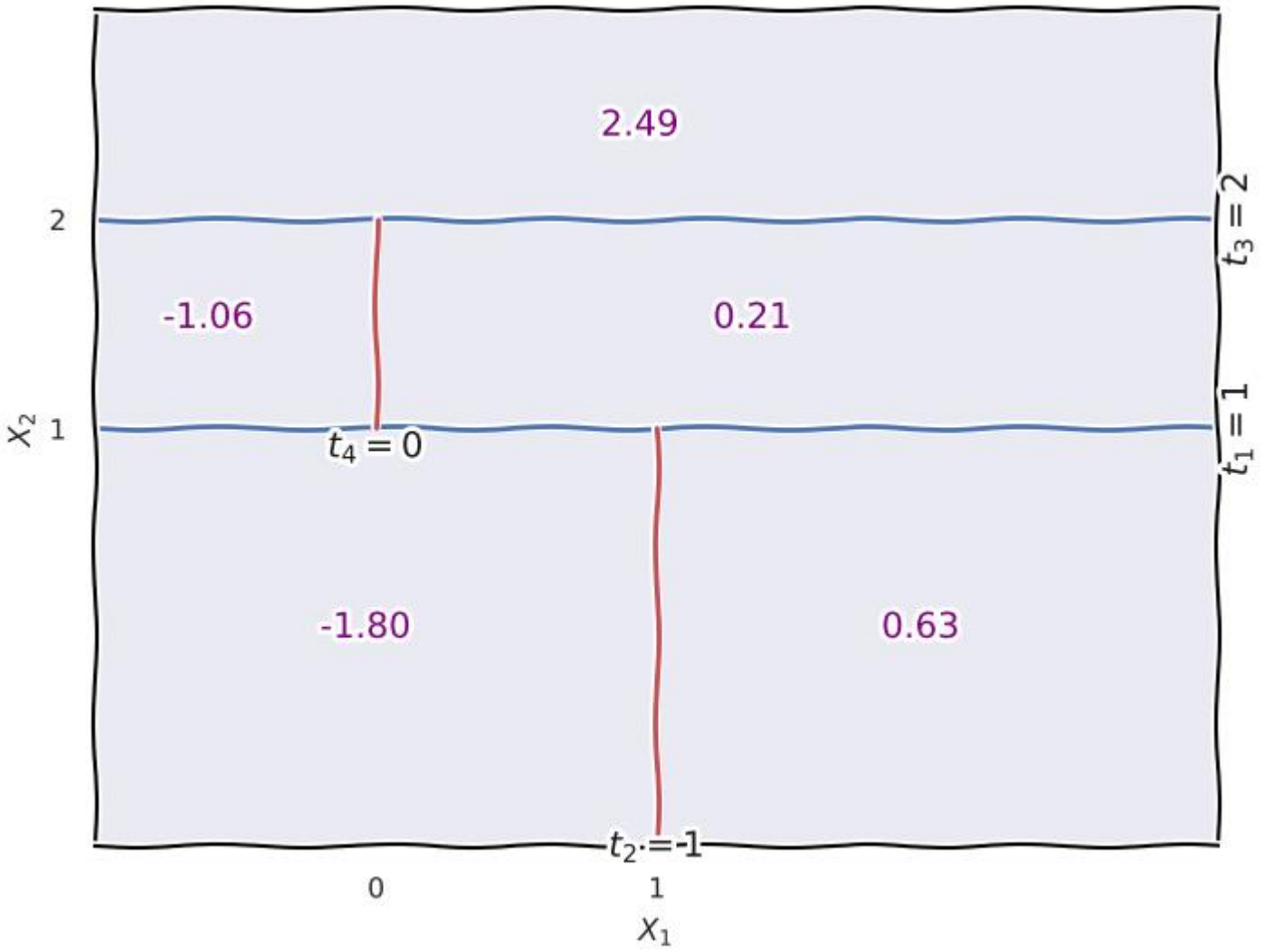




G. James et al., An Introduction to Statistical Learning, Springer Texts in Statistics, https://doi.org/10.1007/978-3-031-38747-0_9

DT from predictor space, and *vice versa*





G. James et al., An Introduction to Statistical Learning, Springer Texts in Statistics, https://doi.org/10.1007/978-3-031-38747-0_9

Gini Impurity, Root node

Gini impurity, Choice of root node

Consider building a decision tree to classify the following dataset. The features are Outlook $\in \{\text{Sunny}, \text{Overcast}\}$ and Humidity $\in \{\text{High}, \text{Normal}\}$, and the label is Play $\in \{\text{Yes}, \text{No}\}$:

Outlook	Humidity	Play
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Overcast	Normal	Yes
Sunny	High	No

(a) Compute the Gini impurity, *G(Play),* of the full dataset

(b) Determine and justify the choice of feature for the root node.

Gini impurity, Choice of root node

Outlook	Humidity	Play
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Overcast	Normal	Yes
Sunny	High	No

(a) Compute the Gini impurity, *G(Play)*, of the full dataset

The Gini impurity for a node S with c classes is calculated as:

$$G(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Where p_i is the probability of an item belonging to class i.

Step-by-step Calculation:

- **Total samples (N):** 5
- **Count of 'Yes':** 3
- **Count of 'No':** 2

First, calculate the probabilities of each class in the root node:

- $p(\text{Yes}) = 3/5 = 0.6$
- $p(\text{No}) = 2/5 = 0.4$

Now, plug these into the formula:

$$G(\text{Play}) = 1 - (0.6^2 + 0.4^2)$$

$$G(\text{Play}) = 1 - (0.36 + 0.16)$$

$$G(\text{Play}) = 1 - 0.52 = 0.48$$

Answer for Part 1: The Gini impurity of the full dataset is **0.48**.

Gini impurity, Choice of root node

Outlook	Humidity	Play
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Overcast	Normal	Yes
Sunny	High	No

(b) Determine and justify the choice of feature for the root node.

To determine the root node, we must calculate the weighted average Gini impurity for a split on each available feature (`Outlook` and `Humidity`). The feature that results in the *lowest* weighted Gini impurity (highest Gini gain) becomes the root.

Evaluating Feature 1: Outlook

This feature splits the 5 samples into two subsets: "Sunny" and "Overcast".

- **Subset: Sunny** (3 samples total)
  - Labels: 1 'Yes', 2 'No'
  - $p(\text{Yes}) = 1/3, p(\text{No}) = 2/3$
  - $G(\text{Sunny}) = 1 - \left(\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2\right)$
  - $G(\text{Sunny}) = 1 - \left(\frac{1}{9} + \frac{4}{9}\right) = 1 - \frac{5}{9} = \frac{4}{9} \approx 0.444$
- **Subset: Overcast** (2 samples total)
  - Labels: 2 'Yes', 0 'No'
  - $p(\text{Yes}) = 2/2 = 1, p(\text{No}) = 0$
  - $G(\text{Overcast}) = 1 - (1^2 + 0^2) = 0$ *(This is a pure node)*
- **Weighted Gini Impurity for Outlook:**

$$G(S, \text{Outlook}) = \left(\frac{3}{5} \times G(\text{Sunny})\right) + \left(\frac{2}{5} \times G(\text{Overcast})\right)$$

$$G(S, \text{Outlook}) = \left(\frac{3}{5} \times \frac{4}{9}\right) + \left(\frac{2}{5} \times 0\right) = \frac{12}{45} = \frac{4}{15} \approx 0.267$$

Gini impurity, Choice of root node

Outlook	Humidity	Play
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Overcast	Normal	Yes
Sunny	High	No

(b) Determine and justify the choice of feature for the root node.

To determine the root node, we must calculate the weighted average Gini impurity for a split on each available feature (`Outlook` and `Humidity`). The feature that results in the *lowest* weighted Gini impurity (highest Gini gain) becomes the root.

Evaluating Feature 2: Humidity

This feature splits the 5 samples into two subsets: "High" and "Normal".

- **Subset: High** (3 samples total)
  - Labels: 1 'Yes', 2 'No' (Notice this is identical to the Sunny subset above)
  - $p(\text{Yes}) = 1/3, p(\text{No}) = 2/3$
  - $G(\text{High}) = \frac{4}{9} \approx 0.444$
- **Subset: Normal** (2 samples total)
  - Labels: 2 'Yes', 0 'No' (Notice this is identical to the Overcast subset above)
  - $p(\text{Yes}) = 2/2 = 1, p(\text{No}) = 0$
  - $G(\text{Normal}) = 0$ *(This is a pure node)*
- **Weighted Gini Impurity for Humidity:**

$$G(S, \text{Humidity}) = \left(\frac{3}{5} \times G(\text{High})\right) + \left(\frac{2}{5} \times G(\text{Normal})\right)$$

$$G(S, \text{Humidity}) = \left(\frac{3}{5} \times \frac{4}{9}\right) + \left(\frac{2}{5} \times 0\right) = \frac{12}{45} = \frac{4}{15} \approx 0.267$$

Gini impurity, Choice of root node

Outlook	Humidity	Play
Sunny	High	No
Sunny	Normal	Yes
Overcast	High	Yes
Overcast	Normal	Yes
Sunny	High	No

(b) Determine and justify the choice of feature for the root node.

To determine the root node, we must calculate the weighted average Gini impurity for a split on each available feature (`Outlook` and `Humidity`). The feature that results in the *lowest* weighted Gini impurity (highest Gini gain) becomes the root.

Conclusion & Justification:

The weighted Gini impurity for splitting on `Outlook` is ≈ 0.267, and the weighted Gini impurity for splitting on `Humidity` is also ≈ 0.267.

Because both features result in the exact same reduction of impurity, there is a **mathematical tie**. You can choose **either** `Outlook` or `Humidity` as the root node. In standard algorithm implementations like `scikit-learn` , the tie is broken arbitrarily or based on the order the features are fed into the model.

Decision Tree example

Decision Tree, Feature selection

Construct the decision tree to help determine whether a child should go out to play.

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

Decision Tree, Feature selection

1. Definitions and Formulas

- **Target Variable:** `Play?` (Possible classes: `Yes`, `No`)
- **Total Samples (N):** 10
- **Target Distribution:**
  - `Yes`: 5 days (Days $2, 3, 4, 7, 9$)
  - `No`: 5 days (Days $1, 5, 6, 8, 10$)

Gini Impurity Formula for a Node (S):

$$G(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Where c is the number of classes (here, 2), and p_i is the probability of class i (e.g., probability of 'Yes'). A Gini value of 0 means a pure node (all samples belong to one class). A Gini value of 0.5 (for binary classification) means maximum impurity (samples are evenly split between classes).

Weighted Average Gini Impurity for a Split on Feature (A):

$$G(S, A) = \sum_{j=1}^{v} \frac{|S_j|}{|S|} G(S_j)$$

Where v is the number of distinct values of feature A, S_j is the subset of samples for value j, and $|S|$ is the total number of samples in the parent node. **Our goal is to minimize this value at each split.**

Decision Tree, Feature selection

Step 0: Calculate Base Gini Impurity (Root Node)

Step 1: Determine the Root Split

Summary of Step 1:

- Weighted Gini for Weather: 0.2835 (Lowest)
- Weighted Gini for Temperature: 0.44
- Weighted Gini for Humidity: 0.4758
- Weighted Gini for Wind: 0.417

We select **Weather** as the split for the root node.

Step 2: Sub-node Splits

Now we examine the three branches created by splitting on `Weather`:

Decision Tree, Feature selection

Step 0: Calculate Base Gini Impurity (Root Node)

Samples: 10 total (5 Yes, 5 No).

$$p(\text{Yes}) = 5/10 = 0.5$$

$$p(\text{No}) = 5/10 = 0.5$$

$$G_{\text{root}} = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = \mathbf{0.50}$$

Decision Tree, Feature selection

Step 1: Determine the Root Split

We need to calculate the weighted Gini impurity for each of the four features (`Weather`, `Temperature`, `Humidity`, `Wind`) to find the best first split.

Feature A: Weather

Values: `Sunny` (Days $1, 3, 8$), `Cloudy` (Days $2, 4, 9$), `Rainy` (Days $5, 6, 7, 10$)

- **Sunny Subset** (3 samples):
  - Labels: 1 Yes (Day 3), 2 No (Days $1, 8$)
  - $G_{\text{Sunny}} = 1 - [(1/3)^2 + (2/3)^2] = 1 - [0.111 + 0.444] = \mathbf{0.445}$
- **Cloudy Subset** (3 samples):
  - Labels: 3 Yes (Days $2, 4, 9$), 0 No
  - This is a pure node. Labels are 100% 'Yes'.
  - $G_{\text{Cloudy}} = 1 - [(3/3)^2 + (0/3)^2] = 1 - 1 = \mathbf{0}$
- **Rainy Subset** (4 samples):
  - Labels: 1 Yes (Day 7), 3 No (Days $5, 6, 10$)
  - $G_{\text{Rainy}} = 1 - [(1/4)^2 + (3/4)^2] = 1 - [0.0625 + 0.5625] = 1 - 0.625 = \mathbf{0.375}$

Weighted Gini Impurity for Weather:

$$G(S, \text{Weather}) = (3/10) \cdot 0.445 + (3/10) \cdot 0 + (4/10) \cdot 0.375$$

$$= 0.1335 + 0 + 0.15 = \mathbf{0.2835}$$

Decision Tree, Feature selection

Step 1: Determine the Root Split

We need to calculate the weighted Gini impurity for each of the four features (`Weather` , `Temperature` , `Humidity` , `Wind`) to find the best first split.

Feature B: Temperature

Values: `Hot` (Days $1, 2, 8, 9$), `Mild` (Days $3, 4, 5, 7, 10$), `Cool` (Day 6)

- **Hot Subset** (4 samples):
  - Labels: 2 Yes (Days $2, 9$), 2 No (Days $1, 8$)
  - $G_{\text{Hot}} = 1 - [(2/4)^2 + (2/4)^2] = 1 - (0.25 + 0.25) = \mathbf{0.50}$
- **Mild Subset** (5 samples):
  - Labels: 3 Yes (Days $3, 4, 7$), 2 No (Days $5, 10$)
  - $G_{\text{Mild}} = 1 - [(3/5)^2 + (2/5)^2] = 1 - (0.36 + 0.16) = 1 - 0.52 = \mathbf{0.48}$
- **Cool Subset** (1 sample):
  - Label: 1 No (Day 6). Pure node.
  - $G_{\text{Cool}} = \mathbf{0}$

Weighted Gini Impurity for Temperature:

$$G(S, \text{Temperature}) = (4/10) \cdot 0.50 + (5/10) \cdot 0.48 + (1/10) \cdot 0$$

$$= 0.20 + 0.24 + 0 = \mathbf{0.44}$$

Decision Tree, Feature selection

Step 1: Determine the Root Split

We need to calculate the weighted Gini impurity for each of the four features (`Weather`, `Temperature`, `Humidity`, `Wind`) to find the best first split.

Feature C: Humidity

Values: `High` (Days $1, 2, 4, 5, 7, 8, 10$), `Normal` (Days $3, 6, 9$)

- **High Subset** (7 samples):
  - Labels: 3 Yes (Days $2, 4, 7$), 4 No (Days $1, 5, 8, 10$)
  - $G_{\text{High}} = 1 - [(3/7)^2 + (4/7)^2] \approx 1 - [0.184 + 0.327] = \mathbf{0.489}$
- **Normal Subset** (3 samples):
  - Labels: 2 Yes (Days $3, 9$), 1 No (Day 6)
  - $G_{\text{Normal}} = 1 - [(2/3)^2 + (1/3)^2] \approx 1 - [0.444 + 0.111] = \mathbf{0.445}$

Weighted Gini Impurity for Humidity:

$$G(S, \text{Humidity}) = (7/10) \cdot 0.489 + (3/10) \cdot 0.445$$

$$= 0.3423 + 0.1335 = \mathbf{0.4758}$$

Decision Tree, Feature selection

Step 1: Determine the Root Split

We need to calculate the weighted Gini impurity for each of the four features (`Weather`, `Temperature`, `Humidity`, `Wind`) to find the best first split.

Feature D: Wind

Values: `Weak` (Days $1, 2, 7, 9$), `Strong` (Days $3, 4, 5, 6, 8, 10$)

- **Weak Subset** (4 samples):
  - Labels: 3 Yes (Days $2, 7, 9$), 1 No (Day 1)
  - $G_{\text{Weak}} = 1 - [(3/4)^2 + (1/4)^2] = 1 - [0.5625 + 0.0625] = \mathbf{0.375}$
- **Strong Subset** (6 samples):
  - Labels: 2 Yes (Days $3, 4$), 4 No (Days $5, 6, 8, 10$)
  - $G_{\text{Strong}} = 1 - [(2/6)^2 + (4/6)^2] \approx 1 - [0.111 + 0.444] = \mathbf{0.445}$

Weighted Gini Impurity for Wind:

$$G(S, \text{Wind}) = (4/10) \cdot 0.375 + (6/10) \cdot 0.445$$

$$= 0.15 + 0.267 = \mathbf{0.417}$$

Decision Tree, Feature selection

Step 1: Determine the Root Split

We need to calculate the weighted Gini impurity for each of the four features (`Weather`, `Temperature`, `Humidity`, `Wind`) to find the best first split.

Summary of Step 1:

- Weighted Gini for Weather: 0.2835 (Lowest)
- Weighted Gini for Temperature: 0.44
- Weighted Gini for Humidity: 0.4758
- Weighted Gini for Wind: 0.417

We select **Weather** as the split for the root node.

Decision Tree, Feature selection

Step 2: Sub-node Splits

Now we examine the three branches created by splitting on `Weather`:

Branch 1: Weather = Cloudy (Days $2, 4, 9$)

Subset contains 3 samples, all belonging to class `Yes`.
This is a pure node. It becomes a leaf node predicting `Play? = Yes`.

Branch 2: Weather = Sunny (Days $1, 3, 8$)

Subset samples: 3 total (1 Yes, 2 No).

- Yes: Day 3
- No: Days $1, 8$

This node is impure. We need to split it using the remaining available features (`Temperature`, `Humidity`, `Wind`).

- **Test split on Humidity for Sunny branch:**
  - Values in Sunny subset: `High` (Days $1, 8$), `Normal` (Day 3)
  - Subset for Humidity=`High` (Sunny): Labels are 2 No (pure). Gini=0.
  - Subset for Humidity=`Normal` (Sunny): Label is 1 Yes (pure). Gini=0.
  - Weighted Gini = $(2/3) \cdot 0 + (1/3) \cdot 0 = \mathbf{0}$. This provides a perfect separation.
- *(Optional sanity check: Spitting on Temperature gives same perfect separation, while Wind gives $1/3 \cdot 0 + 2/3 \cdot 0.5 = 0.33$. Let's use Humidity as it creates a distinct separation point often linked to weather conditions).*

We split the Sunny node on **Humidity**.

Decision Tree, Feature selection

Step 2: Sub-node Splits

Now we examine the three branches created by splitting on `Weather`:

Branch 3: Weather = Rainy (Days $5, 6, 7, 10$)

Subset samples: 4 total (1 Yes, 3 No).

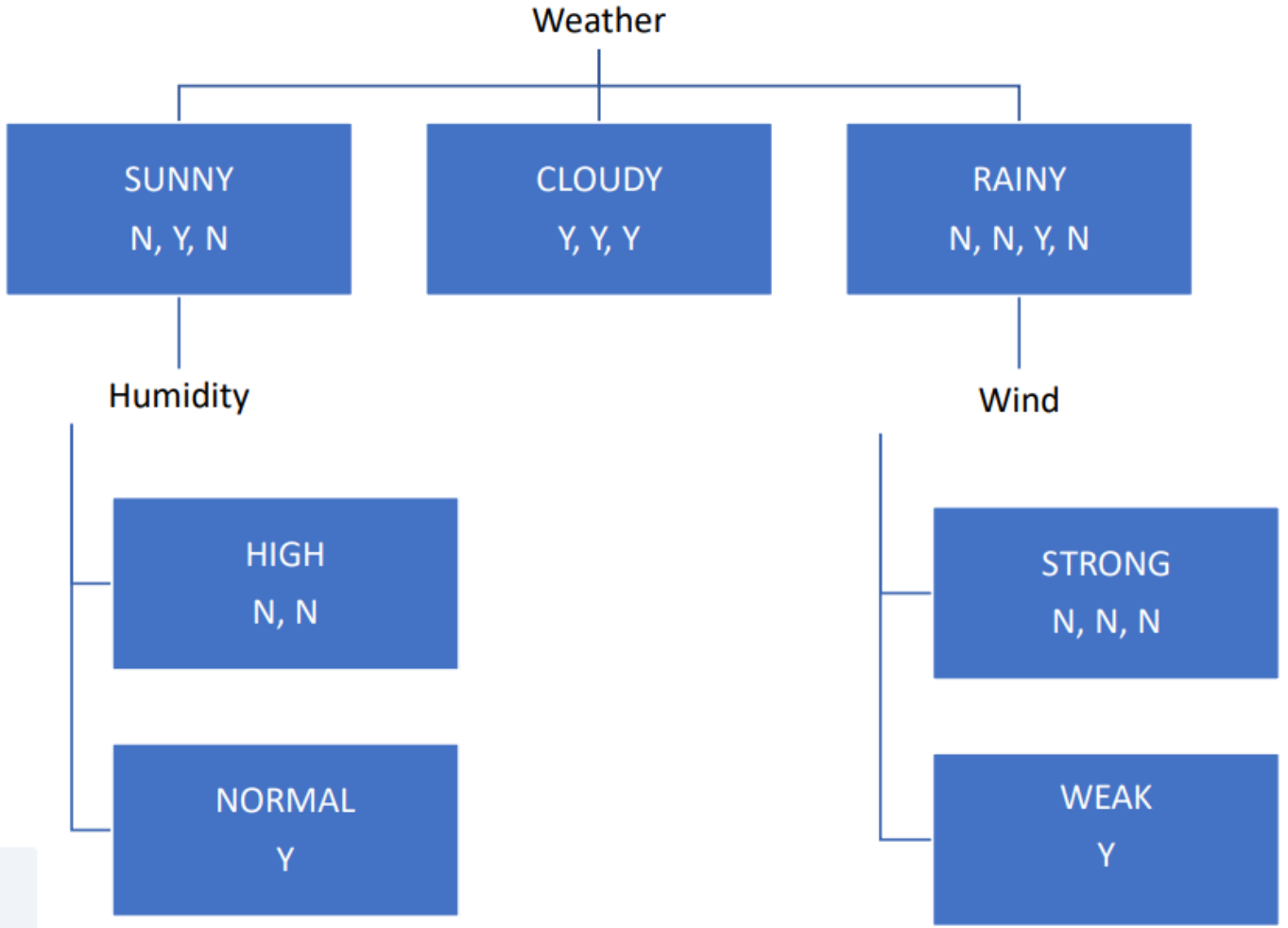
- Yes: Day 7
- No: Days $5, 6, 10$

This node is impure. We need to split it using the remaining available features (`Temperature`, `Humidity`, `Wind`).

- **Test split on Wind for Rainy branch:**
  - Values in Rainy subset: `Weak` (Day 7), `Strong` (Days $5, 6, 10$)
  - Subset for Wind=`Weak` (Rainy): Label is 1 Yes (pure). Gini=0.
  - Subset for Wind=`Strong` (Rainy): Labels are 3 No (pure). Gini=0.
  - Weighted Gini = $(1/4) \cdot 0 + (3/4) \cdot 0 = \mathbf{0}$. This provides a perfect separation.

We split the Rainy node on **Wind**.

Decision Tree, Feature selection




```
Root: [Weather]
  ├── Sunny -> [Humidity]
  │       ├── High -> Play? = No (100% pure, 2 samples: Days 1, 8)
  │       └── Normal -> Play? = Yes (100% pure, 1 sample: Day 3)
  ├── Cloudy -> Play? = Yes (100% pure, 3 samples: Days 2, 4, 9)
  └── Rainy -> [Wind]
          ├── Weak -> Play? = Yes (100% pure, 1 sample: Day 7)
          └── Strong -> Play? = No (100% pure, 3 samples: Days 5, 6, 10)
```

Revision of concepts

Revision of concepts

A decision tree is grown to full depth on a training set of 100 examples and achieves 100% training accuracy.

(a) Why might such a fully grown tree perform poorly on unseen test data? What phenomenon does this illustrate?

(b) Describe two strategies (other than pruning) to mitigate this issue when using decision trees in practice.

(c) Explain how the Random Forest algorithm addresses the overfitting problem in decision trees, mentioning both *bagging* and *feature randomization*.

Revision of concepts

A decision tree is grown to full depth on a training set of 100 examples and achieves 100% training accuracy.

(a) Why might such a fully grown tree perform poorly on unseen test data? What phenomenon does this illustrate?

A decision tree that is grown to full depth until it hits 100% training accuracy has likely memorized the training set completely. Instead of learning the underlying general patterns, it has created highly specific, complex rules to perfectly classify every single data point—including random noise, outliers, and errors specific to that training set.

Because its decision boundaries are overly tailored to the training data, it will fail to generalize well to new, unseen test data. This phenomenon is known as **overfitting** (or having high variance).

Revision of concepts

(b) Describe two strategies (other than pruning) to mitigate this issue when using decision trees in practice.

To prevent a single decision tree from overfitting without using post-training pruning, you can restrict how the tree grows in the first place (often called "pre-pruning" or setting early stopping criteria). Two common strategies are:

1. **Setting a Maximum Depth (`max_depth`):** You can cap the number of levels the tree is allowed to grow. By forcing the tree to stop at a shallow depth (e.g., 3 or 4 levels), you prevent it from creating the hyper-specific, fragmented leaf nodes that memorize noise.
2. **Setting Minimum Samples for Splits/Leaves (`min_samples_split` or `min_samples_leaf`):** You can require that a node must contain at least a certain number of samples (e.g., 5 or 10) to be allowed to split further, or that every leaf node must end up with a minimum number of samples. This ensures the tree only makes rules that apply to a meaningful group of data points, rather than isolating single outliers.

Revision of concepts

(c) Explain how the Random Forest algorithm addresses the over-fitting problem in decision trees, mentioning both *bagging* and *feature randomization*.

Random Forest solves the overfitting problem of single decision trees by building an ensemble (a "crowd") of trees and leveraging two distinct types of randomness to ensure they don't all make the same mistakes.

- **Bagging (Bootstrap Aggregating):** Instead of training one tree on the entire dataset, a Random Forest trains hundreds of unconstrained, fully-grown trees on different random samples of the training data (drawn with replacement). While each individual tree will overfit its specific sample, they will all overfit in slightly different ways. When you average their predictions together, the specific noise learned by individual trees cancels out, drastically reducing the overall variance.
- **Feature Randomization (Decorrelation):** If there is one extremely strong predictive feature in the dataset, standard bagging might result in 100 trees that all use that same feature for their first split, making the trees highly correlated. Feature randomization solves this by forcing each tree to only consider a random subset of features (often $\sqrt{\text{total features}}$) at every single split. This forces the trees to be structurally diverse and "decorrelated," making the final averaged prediction much more robust and generalized.

Thank you for your time!